

ETERNAMENTE

Plataforma Web de Planificación Integral de Bodas — Wedding Planner Digital

IES Portada Alta Ciclo Superior de Desarrollo de Aplicaciones Web — 2º DAW Proyecto Intermodular —
Curso 2025/2026 **Esmeralda López Rodero**

ÍNDICE

1. Introducción y objetivos
 2. Público objetivo
 3. Estado del arte
 4. Metodología, herramientas y tecnologías
 5. Planificación y cronograma
 6. Arquitectura del sistema
 7. Modelo de datos
 8. Flujos de usuario y lógica de navegación
 9. API REST: endpoints
 10. Desarrollo e implementación
 11. Manual de uso
 12. Despliegue
 13. Presupuesto
 14. Bibliografía
 15. Anexo: prototipo Figma
-

1. INTRODUCCIÓN Y OBJETIVOS

1.1 Introducción

El proyecto, denominado **Eternamente**, consiste en el diseño y desarrollo de una plataforma web dirigida a una agencia de organización de bodas. Su finalidad es digitalizar y centralizar la relación entre la pareja, la coordinadora y los invitados para facilitar su organización.

El sector de la organización de eventos nupciales vive actualmente un proceso de transformación digital. Las parejas buscan centralizar en un único entorno todas las decisiones que implica una boda: ceremonia, proveedores, invitados, presupuesto, decoración y comunicación con la coordinadora. Sin embargo, las herramientas disponibles en el mercado solo cubren parte del proceso y presentan carencias significativas: presupuestos poco flexibles y ausencia de comunicación en tiempo real.

Eternamente nace para cubrir esas carencias mediante una aplicación moderna, especializada, visualmente elegante y completamente funcional, donde la pareja gestiona su boda paso a paso con el apoyo directo de una wedding planner.

1.2 Objetivos específicos

Funcionales

- **Cuestionario inicial guiado** de 4 pasos (ceremonia, invitados, estilo, presupuesto) que recoge los datos básicos para crear la boda.
- **Panel de la pareja** con resumen, edición de detalles, organización de mesas, gestión de invitados, selección de proveedores y chat con la coordinadora.
- **Organizador de mesas drag & drop**: asignación visual de invitados a mesas con control de capacidad.
- **Presupuesto dinámico**: cálculo automático del coste estimado a partir de los proveedores seleccionados y solicitud de presupuesto definitivo a la wedding planner.
- **Chat asíncrono** entre cliente y coordinadora persistido en base de datos, con marca de leído/no leído.
- **Panel de administración** completo: dashboard con estadísticas, gestión de clientes y bodas (cambio de estado), CRUD del catálogo (proveedores, paquetes, portfolio, testimonios), bandeja de mensajes de contacto y chat centralizado.
- **Verificación de email** obligatoria mediante enlace firmado antes de poder rellenar el cuestionario.

Técnicos

- Arquitectura cliente-servidor desacoplada: API REST en Laravel 12 consumida desde Angular 21 con interceptores de autenticación.
- Base de datos relacional MySQL 8 con todos los nombres de tablas, columnas y código en español.
- Contenerización completa con Docker Compose (frontend, backend, MySQL, phpMyAdmin, MailHog) para despliegue reproducible.
- Diseño responsive mobile-first mediante CSS3 puro (sin frameworks pesados).
- Autenticación basada en tokens con Laravel Sanctum, middleware de roles y de estado de boda.

2. PÚBLICO OBJETIVO

2.1 Usuario cliente: la pareja

El perfil principal son parejas en proceso de planificación de su boda:

- **Edad**: entre 25 y 55 años, con mayor concentración entre los 28 y 40.
- **Perfil digital**: usuarios habituales de aplicaciones web y móviles. Valoran usabilidad, disponibilidad y seguridad.
- **Necesidades**: centralizar la gestión de todos los elementos (invitados, proveedores, presupuesto, fechas), comunicarse fácilmente con la coordinadora y tener control visual del estado en todo momento.
- **Comportamiento**: acceden desde móvil y tablet en horario tarde/noche y desde escritorio para tareas complejas como el plano de mesas.
- **Motivación**: reducir el estrés del proceso y disfrutar de la planificación con apoyo profesional.

2.2 Usuario administrador: la wedding planner

- **Perfil**: profesional del sector con competencias digitales básico-intermedias. Gestiona varias bodas a la vez.
- **Necesidades**: panel centralizado con el estado de todas las bodas, comunicación con cada cliente, gestión del catálogo de proveedores y control de presupuestos.
- **Acceso**: principalmente desde ordenador en horario laboral.

3. ESTADO DEL ARTE

3.1 ¿Se ha hecho algo similar?

Sí. Existen plataformas que cubren parcialmente la planificación nupcial digital. Las más relevantes son **Zankyou**, **Bodas.net**, **The Knot** y **WeddingWire**. A nivel genérico, herramientas como **Trello** o **Notion** se usan adaptadas para este fin.

Ninguna de ellas ofrece de forma integrada las tres características que definen la propuesta de valor de Eternamente: configurador visual paso a paso, presupuesto dinámico en tiempo real y coordinadora integrada con chat directo.

3.2 Tabla comparativa

Solución	Descripción	Tecnologías	Puntos fuertes	Puntos débiles
Zankyou / Bodas.net	Portales con directorio de proveedores y gestión básica de invitados	PHP, JavaScript, MySQL	Gran base de proveedores, visibilidad de mercado	Presupuestos rígidos, poca interacción, orientado a publicidad
The Knot / WeddingWire	Plataformas líderes con checklists y herramientas de presupuesto	React/Vue, microservicios	Planificación completa, gestión de asientos	Sin tiempo real, presupuesto no dinámico, sin español
Trello / Asana	Gestión de proyectos adaptada a bodas	React/Angular, Node.js	Flexibilidad total, colaboración	Sin módulo nupcial, curva de aprendizaje
Eternamente (propuesta)	Plataforma integral con configurador visual, presupuesto dinámico, chat e invitaciones digitales	Angular 21, Laravel 12, MySQL, Docker	Todo en uno, coordinadora integrada, diseño elegante	Proyecto nuevo, sin histórico de usuarios

3.3 Limitaciones detectadas

- **Presupuestos rígidos:** plantillas fijas o listas manuales sin cálculo automático.
- **Escasa participación de invitados:** sólo RSVP básico.
- **Sin coordinadora integrada:** ningún canal nativo con un profesional.
- **Interfaces poco atractivas:** priorizan funcionalidad sobre experiencia visual.
- **Sin despliegue propio:** las plataformas de terceros no permiten personalización.

3.4 ¿Qué aporta Eternamente?

- Cuestionario inicial guiado para evitar la sensación de caos.
- Presupuesto dinámico con desglose por categorías.

- Plano de mesas interactivo con asignación drag & drop.
- Chat persistente entre la pareja y la coordinadora.
- Despliegue propio y control completo de la infraestructura.

4. METODOLOGÍA, HERRAMIENTAS Y TECNOLOGÍAS

4.1 Metodología

Se adopta una metodología ágil basada en **Scrum** adaptada a un proyecto individual. La elección responde a la necesidad de trabajar por incrementos funcionales, revisar decisiones de forma continua y mantener flexibilidad ante cambios.

- Iteraciones cortas (sprints) con entregables verificables.
- Backlog priorizado y descomposición en historias de usuario.
- Revisión continua y reajuste según el progreso.
- Foco en valor funcional antes que en documentación rígida.

4.2 Adaptación de Scrum

Rol simulado	Responsabilidad	Aplicación
Product Owner	Definir alcance y prioridades	La autora valida funcionalidades y prioriza el backlog
Scrum Master	Organizar la dinámica	Seguimiento semanal, control de riesgos
Frontend Developer	UI y UX	Angular 21 + CSS3
Backend Developer	API, lógica y seguridad	Laravel 12, validaciones, acceso a datos
DevOps / QA	Entornos, pruebas, despliegue	Docker, automatización, calidad, documentación

4.3 Tecnologías y herramientas

Capa	Tecnología
Gestión del proyecto	JIRA
Prototipo	Figma
Frontend	Angular 21 + CSS3 (signals, standalone components)
Backend	Laravel 12 (PHP 8.3)
Autenticación	Laravel Sanctum (tokens personales)
Base de datos	MySQL 8
Email (desarrollo)	MailHog
Infraestructura	Docker + Docker Compose

Capa	Tecnología
Control de versiones	Git + GitHub
Editor	VS Code

5. PLANIFICACIÓN Y CRONOGRAMA

5.1 Estrategia temporal

La planificación se estructura en fases y sprints consecutivos. El tablero detallado se gestiona en Jira.

Sprint / Fase	Objetivo principal	Entregable
Fase 0	Análisis, benchmarking y definición de alcance	Documento base y backlog inicial
Sprint 1	Arquitectura, base del proyecto y autenticación	Estructura Angular/Laravel operativa, login/registro
Sprint 2	Cuestionario y catálogo de proveedores	Módulo de planificación funcional
Sprint 3	Invitados y presupuesto dinámico	Gestión social y económica integrada
Sprint 4	Mesas e invitaciones	Seating plan y comunicación
Sprint 5	Chat, notificaciones y panel admin	Comunicación y supervisión
Sprint 6	Pruebas, despliegue y documentación	Versión candidata a entrega

Las capturas del tablero Jira se incluyen en el anexo.

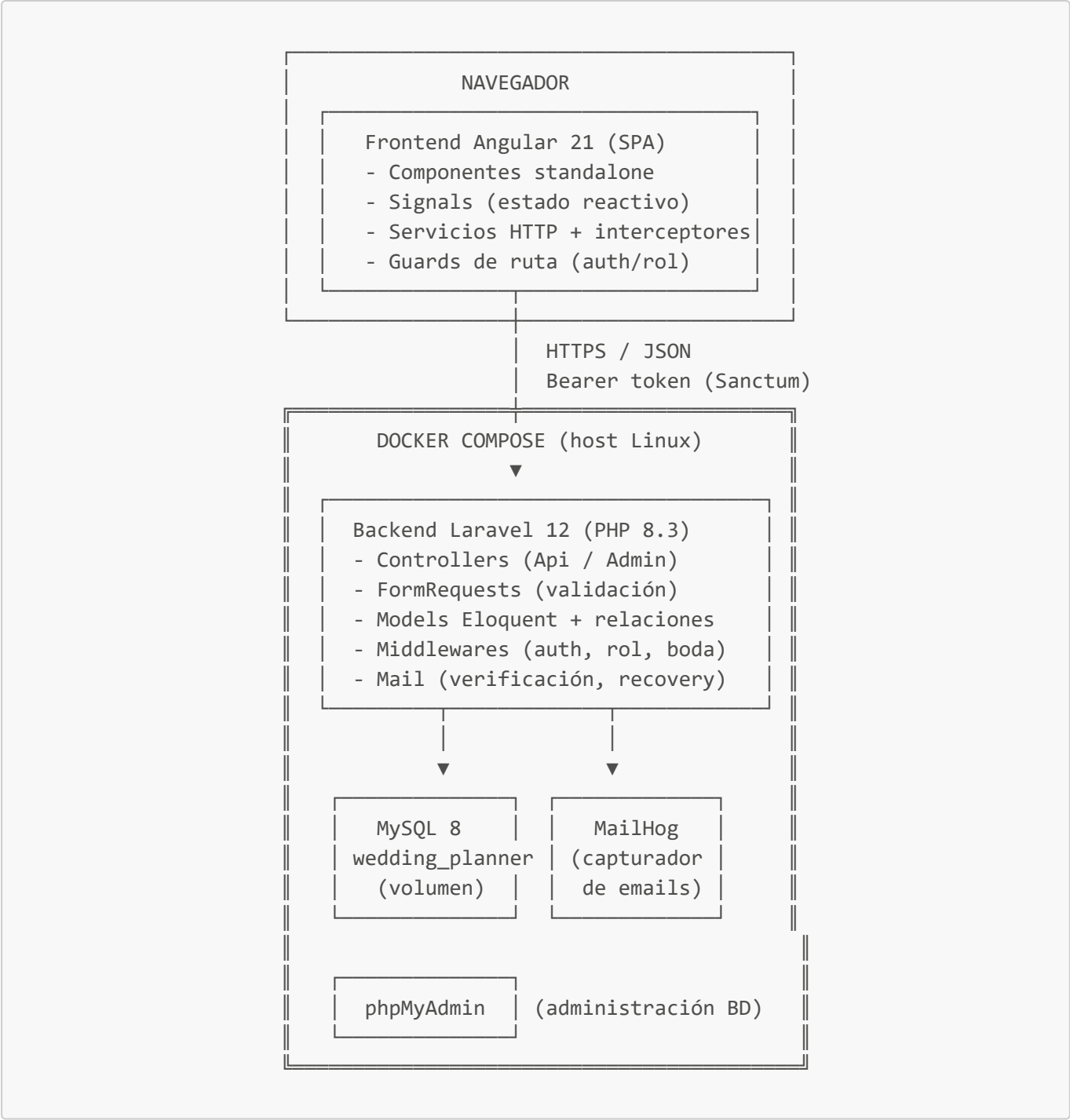
5.2 Hitos relevantes

- Validación del alcance funcional y del problema a resolver.
- Definición de arquitectura y modelo de datos.
- Entrega del primer incremento navegable con autenticación.
- Integración del presupuesto dinámico con el cuestionario.
- Disponibilidad del módulo de mesas e invitados.
- Integración del chat y panel admin.
- Redacción y revisión completa de la memoria.

6. ARQUITECTURA DEL SISTEMA

6.1 Visión general

La aplicación sigue una arquitectura **cliente-servidor desacoplada** sobre contenedores Docker. El frontend (SPA Angular) se comunica con el backend (API REST Laravel) mediante HTTP/JSON, y el backend persiste en MySQL.



6.2 Servicios Docker

Servicio	Imagen	Puerto	Función
frontend	Node 20 + Angular CLI	4200	ng serve con hot reload
backend	PHP 8.3-fpm + Composer	8000	API Laravel con php artisan serve
mysql	mysql:8	3306	Base de datos persistente (volumen mysql_datos)
phpmyadmin	phpmyadmin:5	8080	Gestor visual de la BD

Servicio	Imagen	Puerto	Función
mailhog	mailhog/mailhog	1025 / 8025	Captura emails en desarrollo

Todos los servicios viven en una red Docker interna (`wp_red`). El frontend depende del backend; el backend depende de MySQL.

6.3 Estructura de carpetas

```

ProyectoIntermodular/
├── docker-compose.yml
├── README.md
├── frontend/                                ← Angular 21
│   ├── src/app/
│   │   ├── componentes/                    ← Reutilizables (navbar, hero, footer...)
│   │   ├── paginas/                        ← Vistas de cada ruta
│   │   │   ├── landing/
│   │   │   ├── login/ registro/ recuperar-password/ email-verificado/
│   │   │   ├── panel-cliente/ ← cuestionario, resumen-boda, mesas, chat...
│   │   │   └── panel-admin/   ← dashboard, clientes, bodas, catálogo...
│   │   ├── servicios/                 ← *.service.ts (HTTP + estado)
│   │   ├── modelos/                   ← Interfaces TypeScript (Boda, Usuario...)
│   │   ├── guards/                    ← Protección de rutas
│   │   └── interceptores/             ← Token Bearer, gestión de 401
│   └── src/styles.scss                 ← Variables CSS y estilos globales
└── backend/                             ← Laravel 12
    ├── app/
    │   ├── Http/Controllers/Api/        ← Cliente
    │   ├── Http/Controllers/Api/Admin/  ← Administrador
    │   ├── Http/Requests/                ← Validación
    │   ├── Http/Middleware/              ← BodaActiva, EsAdministrador...
    │   └── Models/                       ← Eloquent
    ├── database/
    │   ├── migrations/                   ← Esquema
    │   └── seeders/                      ← Datos de prueba
    ├── routes/api.php                    ← Definición de rutas REST
    └── storage/app/public/                ← Imágenes (portfolio, proveedores...)

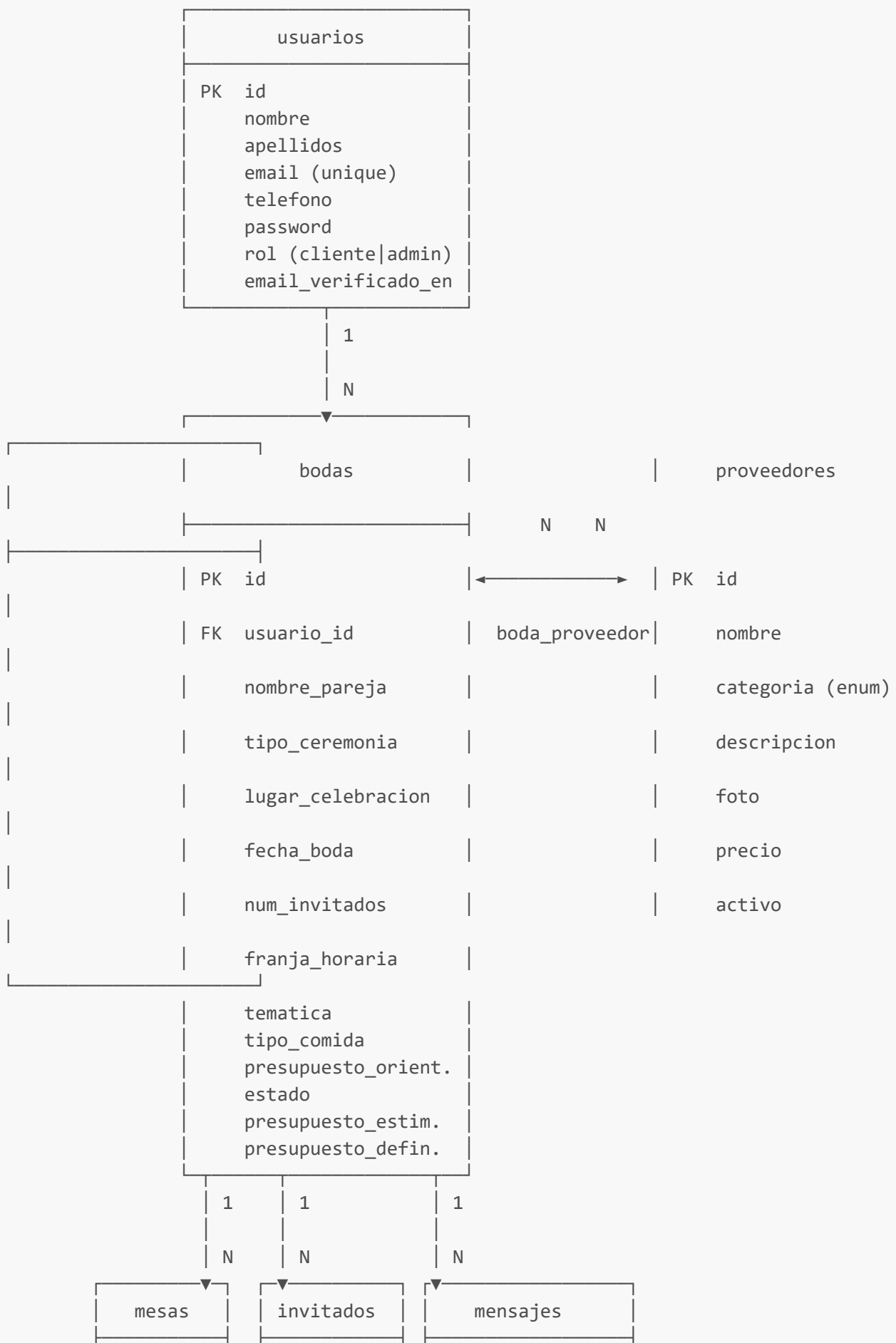
```

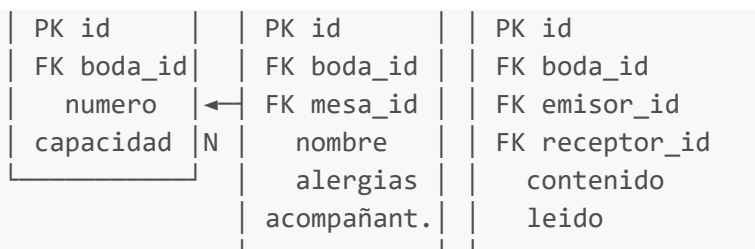
6.4 Patrones aplicados

- **Repository pattern** simplificado a través de Eloquent.
- **Form Request** para mantener controllers delgados.
- **DTO implícito** mediante interfaces TypeScript en frontend.
- **Signals + computed** en componentes Angular para estado reactivo.
- **Service singleton** (`providedIn: 'root'`) que comparte estado entre componentes (p. ej. `BodaService.bodaActual`).

7. MODELO DE DATOS

7.1 Diagrama Entidad-Relación





Tablas independientes del flujo de boda (catálogo público y administración):

paquetes

portfolio

testimonios

mensajes_contacto

7.2 Resumen de tablas

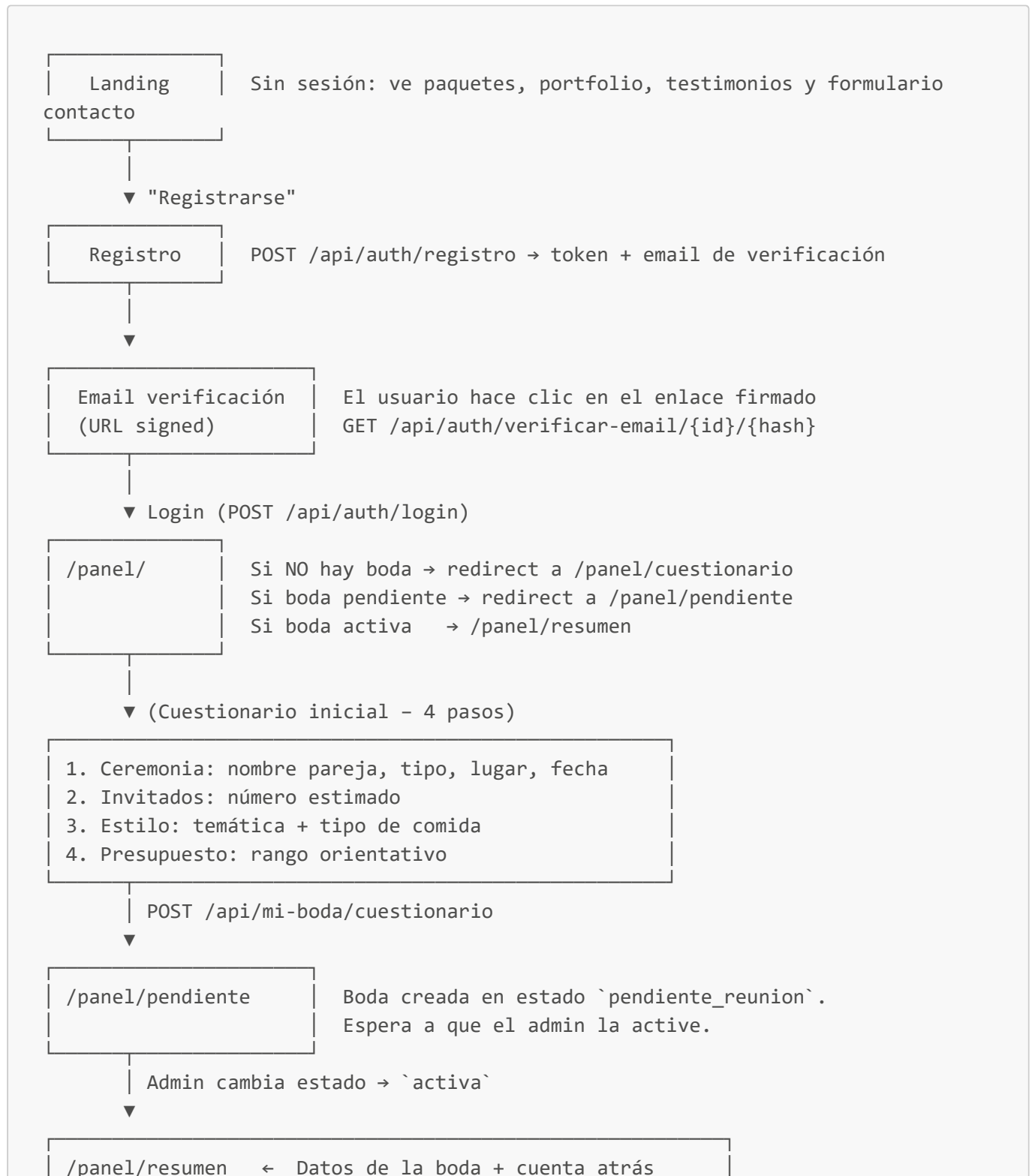
Tabla	Filas-clave	Notas
usuarios	id, email (unique), rol	Roles: cliente / administrador . Verificación de email obligatoria.
bodas	id, usuario_id, estado	Una boda por usuario. Estados: pendiente_reunion → activa → finalizada / cancelada .
proveedores	id, categoria, precio	10 categorías (lugar, floristeria, musica, fotografia, catering, decoracion, vestuario, transporte, detalles, tarta).
boda_proveedor	boda_id, proveedor_id, notas	Tabla pivote N:M. Suma de precios → presupuesto estimado.
mesas	id, boda_id, numero, capacidad	Único por (boda, numero).
invitados	id, boda_id, mesa_id, alergias	mesa_id nullable (invitado sin asignar).
mensajes	id, boda_id, emisor_id, receptor_id	Chat persistente. Índice por (receptor_id , leído).
paquetes	id, nombre, precio	Catálogo público de la landing.
portfolio	id, titulo, foto, orden	Galería pública.
testimonios	id, nombre, mensaje, valoracion	Sección de testimonios.
mensajes_contacto	id, nombre, email, mensaje, leído	Formulario de contacto.

7.3 Restricciones e integridad

- Todas las claves foráneas con `cascadeOnDelete` salvo `invitados.mesa_id` que es `nullOnDelete` (eliminar una mesa no borra invitados, sólo los desasigna).
- ENUM en MySQL para campos cerrados (`tipo_ceremonia`, `lugar_celebracion`, `franja_horaria`, `tematica`, `tipo_comida`, `presupuesto_orientativo`, `estado`, `categoria`, `rol`).
- Índices en `bodas.estado`, `bodas.fecha_boda`, `mensajes.(boda_id, created_at)` y `mensajes.(receptor_id, leído)` para optimizar las consultas frecuentes.

8. FLUJOS DE USUARIO Y LÓGICA DE NAVEGACIÓN

8.1 Flujo del cliente (pareja)

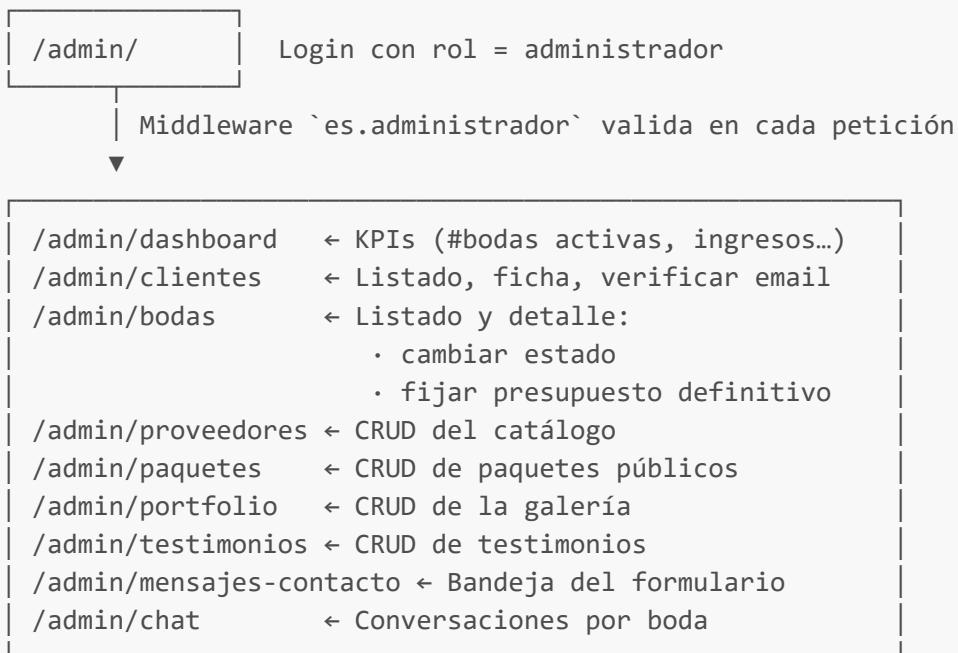


/panel/mesas	← Drag & drop invitados ↔ mesas
/panel/personalizacion	← Selección de proveedores
/panel/presupuesto	← Desglose + solicitar definitivo
/panel/chat	← Conversación con la coordinadora
/panel/perfil	← Editar datos / contraseña

Puntos críticos del flujo (sin puntos muertos):

- Si el usuario **no ha verificado** el email, el cuestionario se bloquea con un mensaje y un botón para reenviar la verificación.
- Si el usuario **ya tiene boda en pendiente_reunion** y vuelve al cuestionario, el endpoint actualiza la existente en lugar de crear duplicado.
- Si el estado es **activa**, las acciones de proveedores/mesas/invitados/chat se desbloquean (middleware **boda.activa**).
- Si es **finalizada** o **cancelada**, los detalles son de sólo lectura.

8.2 Flujo del administrador



8.3 Flujo de gestión de invitados (detalle)

1. El cliente entra en **/panel/mesas**. Frontend dispara **GET /mi-boda/invitados** y **GET /mi-boda/mesas**.
2. La pantalla muestra dos columnas: invitados sin asignar (panel izquierdo) y mesas con sus huecos libres (cuadrícula derecha).
3. Al **arrastrar** un invitado sobre una mesa:
 - El frontend valida en cliente que la mesa tiene plazas (**capacidad - invitados_actuales > 0**).
 - Envía **POST /mi-boda/asignar-mesa** con **{invitado_id, mesa_id}**.

- Si el backend acepta, devuelve el invitado actualizado y el frontend mueve la tarjeta visualmente.
 - Si responde 422 (mesa llena, validación), se muestra el error sin mover la tarjeta.
4. Eliminar un invitado o una mesa:
- `DELETE /mi-boda/invitados/{id}` o `DELETE /mi-boda/mesas/{id}`.
 - Al borrar una mesa, los invitados que tenía pasan a `mesa_id = null` (regla `nullOnDelete`).

8.4 Flujo de chat

1. El cliente abre `/panel/chat`. Se hace `GET /mi-boda/chat` que devuelve la conversación completa (mensajes ordenados por `created_at`).
2. Al cargar, se llama `POST /mi-boda/chat/leer` para marcar como leídos los mensajes recibidos.
3. El cliente envía un mensaje: `POST /mi-boda/chat` con `{contenido}`. El backend resuelve `receptor_id` automáticamente al admin de turno.
4. El admin ve la lista de conversaciones en `/admin/chat`. Al abrir una, se cargan sus mensajes y responde con el endpoint paralelo `POST /admin/chat/{boda}`.

9. API REST: ENDPOINTS

9.1 Convenciones

- Todas las rutas bajo el prefijo `/api`.
- Respuesta estándar:

```
{ "success": true, "data": { ... }, "message": "" }
```

- Autenticación con `Authorization: Bearer <token>` (Sanctum).
- Errores de validación devuelven HTTP 422 con `data = objeto {campo: [mensajes]}`.

9.2 Tabla de endpoints

Método	Ruta	Auth	Descripción
Públicas			
GET	<code>/paquetes</code>	–	Catálogo público de paquetes
GET	<code>/portfolio</code>	–	Galería pública
GET	<code>/testimonios</code>	–	Testimonios
POST	<code>/contacto</code>	–	Mensaje desde formulario de contacto
Autenticación			
POST	<code>/auth/registro</code>	–	Crear cuenta + token + email verificación

Método	Ruta	Auth	Descripción
POST	/auth/login	–	Login (devuelve token)
POST	/auth/recuperar-password	–	Solicitar email de reset
POST	/auth/reset-password	–	Reset con token recibido por email
GET	/auth/verificar-email/{id}/{hash}	signed	Verificación por enlace firmado
GET	/auth/yo	sanctum	Datos del usuario logueado
POST	/auth/logout	sanctum	Revocar token
POST	/auth/reenviar-verificacion	sanctum	Reenviar email
Cliente			
PUT	/mi-perfil	sanctum + verified	Editar perfil
PUT	/mi-perfil/password	sanctum + verified	Cambiar contraseña
GET	/mi-boda	sanctum + verified	Estado y detalles de la boda + cuenta atrás
POST	/mi-boda/cuestionario	sanctum + verified	Crear/actualizar boda inicial
PUT	/mi-boda/detalles	sanctum + verified	Editar detalles de la boda
Cliente con boda activa			
GET	/proveedores	+ boda.activa	Catálogo (para selección)
GET	/proveedores/{categoria}	+ boda.activa	Filtrado por categoría
GET / POST	/mi-boda/proveedores	+ boda.activa	Ver / sincronizar selección
POST	/mi-boda/solicitar-presupuesto	+ boda.activa	Solicitar presupuesto definitivo
GET / POST / PUT / DELETE	/mi-boda/invitados[/]{id}]	+ boda.activa	CRUD invitados

Método	Ruta	Auth	Descripción
GET / POST / PUT / DELETE	/mi-boda/mesas[/]{id}]	+ boda.activa	CRUD mesas
POST	/mi-boda/asignar-mesa	+ boda.activa	Asignar / desasignar invitado
GET / POST	/mi-boda/chat	+ boda.activa	Conversación
POST	/mi-boda/chat/leer	+ boda.activa	Marcar leídos
Administrador			
GET	/admin/dashboard	+ es.administrador	KPIs y estadísticas
GET	/admin/clientes[/]{id}]	+ es.administrador	Listado / ficha de cliente
POST	/admin/clientes/{id}/verificar-email	+ es.administrador	Forzar verificación manual
GET	/admin/bodas[/]{boda}]	+ es.administrador	Listado / detalle
PUT	/admin/bodas/{boda}/estado	+ es.administrador	Cambiar estado
PUT	/admin/bodas/{boda}/presupuesto	+ es.administrador	Fijar presupuesto definitivo
GET / POST / PUT / DELETE	/admin/proveedores[/]{id}]	+ es.administrador	CRUD catálogo
GET / POST / PUT / DELETE	/admin/paquetes[/]{id}]	+ es.administrador	CRUD paquetes
GET / POST / PUT / DELETE	/admin/portfolio[/]{id}]	+ es.administrador	CRUD galería
GET / POST / PUT / DELETE	/admin/testimonios[/]{id}]	+ es.administrador	CRUD testimonios
GET	/admin/mensajes-contacto	+ es.administrador	Bandeja contacto
POST	/admin/mensajes-contacto/{id}/leer	+ es.administrador	Marcar leído
DELETE	/admin/mensajes-contacto/{id}	+ es.administrador	Eliminar
GET	/admin/chat[/]{boda}]	+ es.administrador	Conversaciones

Método	Ruta	Auth	Descripción
POST	/admin/chat/{boda}	+ es.administrador	Enviar mensaje al cliente

9.3 Ejemplo de request/response

Login

```
POST /api/auth/login
Content-Type: application/json

{ "email": "lucia@example.com", "password": "cliente1234" }
```

```
{
  "success": true,
  "data": {
    "usuario": { "id": 2, "nombre": "Lucia", "rol": "cliente",
    "email_verificado_en": "...",
    "token": "12|aBcDeFgHiJkLmNoPqRsTuVwXyZ"
  },
  "message": "Sesión iniciada."
}
```

Mi boda

```
GET /api/mi-boda
Authorization: Bearer 12|aBcDeFgHiJkLmNoPqRsTuVwXyZ
```

```
{
  "success": true,
  "data": {
    "boda": {
      "id": 1, "nombre_pareja": "Lucia y David",
      "tipo_ceremonia": "religiosa", "lugar_celebracion": "iglesia",
      "fecha_boda": "2026-09-15", "num_invitados": 120,
      "estado": "pendiente_reunion", "presupuesto_estimado": "0.00",
      "proveedores": [], "mesas": [], "invitados": []
    },
    "cuenta_atras_dias": 136
  }
}
```

10. DESARROLLO E IMPLEMENTACIÓN

10.1 Fragmentos representativos

Backend — Form Request con validación reutilizable (`CuestionarioInicialRequest.php`):

```
public function rules(): array
{
    return [
        'nombre_pareja'      => ['required', 'string', 'max:150'],
        'tipo_ceremonia'     => ['required',
Rule::in(['religiosa', 'civil_ayuntamiento', 'simbolica', 'renovacion_votos'])],
        'lugar_celebracion' => ['required',
Rule::in(['iglesia', 'ayuntamiento', 'finca', 'playa', 'jardin', 'restaurante', 'otro'])
    ],
        'fecha_boda'        => ['required', 'date', 'after:today'],
        'num_invitados'     => ['required', 'integer', 'min:1', 'max:1000'],
        // ...
    ];
}
```

Backend — Controller delgado (`BodaController::actualizarDetalles`):

```
public function actualizarDetalles(ActualizarDetallesRequest $request)
{
    $boda = $request->user()->bodas()->latest()->first();

    if (in_array($boda->estado, [Boda::ESTADO_FINALIZADA, Boda::ESTADO_CANCELADA],
true)) {
        return $this->ko('No puedes modificar los detalles de una boda finalizada
o cancelada.', 422);
    }

    $boda->update($request->validated());
    return $this->ok(['boda' => $boda-
>fresh(['proveedores', 'mesas', 'invitados'])], 'Detalles actualizados.');
```

Backend — Middleware personalizado (`BodaActiva`):

```
public function handle(Request $request, Closure $next)
{
    $boda = $request->user()->bodas()->latest()->first();
    if (!$boda || !$boda->estaActiva()) {
        return response()->json([
            'success' => false,
            'message' => 'Tu boda aún no está activa.',
        ], 403);
    }
}
```

```

    }
    $request->attributes->set('boda', $boda);
    return $next($request);
}

```

Frontend — Servicio con signal compartida (BodaService):

```

@Injectable({ providedIn: 'root' })
export class BodaService {
  readonly bodaActual = signal<Boda | null>(null);

  miBoda(): Observable<RespuestaApi<RespuestaMiBoda>> {
    return this.http.get<RespuestaApi<RespuestaMiBoda>>(this.base)
      .pipe(tap((r) => this.bodaActual.set(r.data.boda)));
  }

  actualizarDetalles(datos: DatosCuestionarioInicial) {
    return this.http.put<RespuestaApi<{ boda: Boda }>>(`${this.base}/detalles`,
      datos)
      .pipe(tap((r) => this.bodaActual.set(r.data.boda)));
  }
}

```

Frontend — Computed reactivo (PanelCliente):

```

protected boda = this.bodaService.bodaActual;

protected cuentaAtras = computed<number | null>(() => {
  const fecha = this.boda()?.fecha_boda;
  if (!fecha) return null;
  const objetivo = new Date(fecha); objetivo.setHours(0,0,0,0);
  const hoy = new Date(); hoy.setHours(0,0,0,0);
  return Math.max(0, Math.round((objetivo.getTime() - hoy.getTime()) /
    86_400_000));
});

```

Frontend — Interceptor de token (autenticacion.interceptor.ts):

```

export const authTokenInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AutenticacionService).obtenerToken();
  return next(token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req);
};

```

10.2 Convenciones de código

- **Español** en nombres de tablas, columnas, modelos, componentes, servicios y variables (excepto palabras reservadas del framework).
- **Componentes Angular standalone** sin sufijo `.component` (`navbar.ts`, `navbar.html`, `navbar.scss`).
- **Servicios** con sufijo `.service.ts`.
- **Guards** con sufijo `.guard.ts`.
- Comentarios escasos: solo cuando el "porqué" no es obvio.

10.3 Optimizaciones aplicadas

- **Compresión de imágenes del portfolio** (1600 px de ancho, calidad JPEG 80) → reducción del 80–89 % del peso (de 5 MB a 920 KB).
- `@defer (on viewport)` en la sección portfolio de la landing → la galería no instancia componente ni lanza HTTP hasta que el usuario llega.
- `loading="lazy" + decoding="async" + fetchpriority="low"` en imágenes secundarias.
- `prepareBindings` y eager loading (`with(['proveedores', 'mesas', 'invitados'])`) en backend para evitar N+1.
- **Índices** en columnas de filtrado y unión (`bodas.estado`, `mensajes.receptor_id+leido`, ...).

11. MANUAL DE USO

11.1 Acceso y registro

1. La pareja entra en la landing y pulsa **Registrarse**.
2. Rellena nombre, apellidos, email y contraseña.
3. Recibe un email con enlace de verificación firmado (en desarrollo se ve en MailHog → `http://localhost:8025`).
4. Tras hacer clic en el enlace queda verificada y puede iniciar sesión.

11.2 Cuestionario inicial

Al iniciar sesión por primera vez, se redirige al cuestionario de 4 pasos:

- **Paso 1:** nombres de la pareja, tipo de ceremonia (religiosa, civil ayuntamiento/juzgado, simbólica o renovación de votos), lugar (iglesia, ayuntamiento, finca, playa, jardín, restaurante, otro), fecha y franja horaria.
- **Paso 2:** número estimado de invitados (con contador ± 10).
- **Paso 3:** temática (clásica, rústica, moderna, boho o glamour) y tipo de comida (cóctel, banquete, buffet, familiar).
- **Paso 4:** presupuesto orientativo por rangos.

Al finalizar, la boda queda en estado *pendiente de reunión*. La wedding planner se pondrá en contacto y la activará desde su panel.

11.3 Panel del cliente (boda activa)

- **Resumen:** tarjeta con cuenta atrás, datos editables y accesos rápidos.
- **Personalización:** catálogo de proveedores agrupados por categoría. Marca una opción por categoría para añadirla al presupuesto.

- **Mesas:** arrastra un invitado desde el panel izquierdo a una mesa. Crea mesas indicando número y capacidad. Borra una mesa y los invitados quedan libres.
- **Presupuesto:** desglose por categoría con el coste estimado. Botón "Solicitar presupuesto definitivo" envía un mensaje al admin.
- **Chat:** conversación persistida con la coordinadora.
- **Mi perfil:** editar datos personales y contraseña.

11.4 Panel del administrador

- **Dashboard:** número de bodas activas, ingresos previstos y mensajes pendientes.
- **Clientes:** listado con búsqueda y ficha individual.
- **Bodas:** listado por estado. La acción más usada es *cambiar estado* (pendiente_reunion → activa) y *fijar presupuesto definitivo*.
- **Catálogo:** CRUD completo de proveedores, paquetes, portfolio y testimonios.
- **Mensajes de contacto:** bandeja con marca de leído.
- **Chat:** lista de conversaciones por boda; al abrir una, se ven los mensajes y se puede responder.

Las capturas de cada pantalla se incluyen en el anexo del manual.

12. DESPLIEGUE

12.1 Estrategia

El despliegue se realiza con **Docker Compose**, lo que permite empaquetar todos los servicios y mover la solución entre local y servidor con mínima fricción. Sobre un servidor Linux con Nginx (o proxy equivalente) basta con:

1. Clonar el repositorio.
2. Copiar `.env.example` → `.env` y configurar variables sensibles (DB password, APP_KEY, MAIL_*).
3. `docker compose up -d --build`.
4. `docker exec wp_backend php artisan migrate --seed` (sólo en primera puesta en marcha).
5. `docker exec wp_backend php artisan storage:link`.

12.2 Buenas prácticas

- Separación entre configuración sensible (`.env`) y código fuente.
- Backups periódicos del volumen `mysql_datos` y de `storage/app/public`.
- Logs de aplicación (`storage/logs/laravel.log`) y de servidor para trazabilidad.
- Posibilidad de incorporar CI/CD (GitHub Actions) para construir y verificar el proyecto en cada push.

12.3 Mantenimiento

Una vez desplegado, el sistema requiere:

- **Correctivo:** errores detectados en producción.
- **Adaptativo:** cambios de entorno o de versión de dependencias.
- **Evolutivo:** nuevas funcionalidades derivadas del uso real.

13. PRESUPUESTO

13.1 Estimación de costes

Bloque	Horas estimadas	Coste/hora	Subtotal
Análisis y planificación	55	12 €	660 €
Diseño UX/UI y prototipado	45	12 €	540 €
Desarrollo frontend	140	13 €	1.820 €
Desarrollo backend y API	130	13 €	1.690 €
BBDD, Docker y despliegue	45	13 €	585 €
Pruebas, documentación y defensa	65	12 €	780 €
Infraestructura básica (hosting, dominio, backups)	–	–	180 €
Herramientas de pago	–	–	0 € (software libre)
Imprevistos (10 %)	–	–	882 €
TOTAL			7.137 €

El presupuesto tiene carácter orientativo y académico. No representa un coste cerrado de mercado, sino una estimación razonable del esfuerzo requerido para una primera versión funcional del producto.

14. BIBLIOGRAFÍA

- Angular Team. (2024). *Angular 21 Documentation*. <https://angular.dev>
- Laravel LLC. (2024). *Laravel 12 Documentation*. <https://laravel.com/docs>
- Laravel Sanctum. <https://laravel.com/docs/sanctum>
- Docker Inc. (2024). *Docker Compose Documentation*. <https://docs.docker.com/compose>
- Mozilla Developer Network. *CSS Reference*. <https://developer.mozilla.org/es/docs/Web/CSS>
- Zankyou Spain. <https://www.zankyou.es>
- The Knot Worldwide. <https://www.theknot.com>
- WeddingWire. <https://www.weddingwire.com>
- Statista. (2024). *Mercado nupcial en España: estadísticas 2024*. <https://www.statista.com>

15. ANEXO: PROTOTIPO FIGMA

Capturas del prototipo Figma con las pantallas principales (landing, login, panel cliente, panel admin, cuestionario).

Capturas del tablero JIRA con la planificación por sprints.

Capturas del manual de uso por pantalla.